SCAPY

Création de trames, analyse, modification, sniffing

Demarrer



$ sudo scapy -H

Welcome to Scapy (2.4.0)

>>>

Prompt interactif

OTERIA CYBER SCHOOL

```
>>>a=IP(ttl=10)
>>>a
< IP ttl=10 |>
>>>a.src
’127.0.0.1’
>>>a.dst="192.168.1.1"
>>>a
< IP ttl=10 dst=192.168.1.1 |>
>>>a.src
’192.168.8.14’
>>>del(a.ttl)
>>>a
< IP dst=192.168.1.1 |>
>>>a.ttl
64
```

Travailler par couches




```
>>>IP()
<IP |>
>>>IP()/TCP()
<IP frag=0 proto=TCP |<TCP |>>
>>>Ether()/IP()/TCP()
<Ether type=0x800 |<IP frag=0 proto=TCP |<TCP |>>>
>>>IP()/TCP()/"GET / HTTP/1.0\r\n\r\n"
<IP frag=0 proto=TCP |<TCP |<Raw load='GET / HTTP/1.0\r\n\r\n' |>>>
>>>Ether()/IP()/IP()/UDP()
<Ether type=0x800 |<IP frag=0 proto=IP |<IP frag=0 proto=UDP |<UDP |>>>>
>>>IP(proto=55)/TCP()
<IP frag=0 proto=55 |<TCP |>>
```

Travailler avec les couches




```
>>>raw(IP())
'E\x00\x00\x14\x00\x01\x00\x00@\x00|\xe7\x7f\x00\x00\x01\x7f\x00\x00\x01'
>>>IP(_)
<IP version=4L ihl=5L tos=0x0 len=20 id=1 flags= frag=0L ttl=64 proto=IP chksum=0x7ce7
src=127.0.0.1 dst=127.0.0.1 |> >>>a=Ether()/IP(dst="www.oteria.fr")/TCP()/"GET
/index.html HTTP/1.0 \n\n" >>>hexdump(a)
00 02 15 37 A2 44 00 AE F3 52 AA D1 08 00 45 00 ...7.D...R....E.
00 43 00 01 00 00 40 06 78 3C C0 A8 05 15 42 23 .C....@.x<....B#
FA 97 00 14 00 50 00 00 00 00 00 00 00 00 50 02 .....P........P.
20 00 BB 39 00 00 47 45 54 20 2F 69 6E 64 65 78 ..9..GET /index
2E 68 74 6D 6C 20 48 54 54 50 2F 31 2E 30 20 0A .html HTTP/1.0 . 0A
```

Travailler avec les couches




```
>>>b=raw(a)

>>>b
'\x00\x02\x157\xa2D\x00\xae\xf3R\xaa\xd1\x08\x00E\x00\x00C\x00\x01\x00\x00@\x
06x<\xc0\xa8\x05\x15B#\xfa\x97\x00\x14\x00P\x00\x00\x00\x00\x00\x00\x00\x00P
\x02 \x00 \xbb9\x00\x00GET /index.html HTTP/1.0 \n\n’

>>>c=Ether(b)

>>>c

<Ether dst=00:02:15:37:a2:44 src=00:ae:f3:52:aa:d1 type=0x800 |<IP version=4L ihl=5L
tos=0x0 len=67 id=1 flags= frag=0L ttl=64 proto=TCP chksum=0x783c src=192.168.5.21
dst=66.35.250.151 options='' |<TCP sport=20 dport=80 seq=0L ack=0L dataofs=5L
reserved=0L flags=S window=8192 chksum=0xbb39 urgptr=0 options=[] |<Raw load='GET
/index.html HTTP/1.0 \n\n' |>>>>
```

Travailler avec les couches




```
>>>c.hide_defaults()

>>>c

<Ether dst=00:0f:66:56:fa:d2 src=00:ae:f3:52:aa:d1 type=0x800 |<IP ihl=5L len=67
frag=0 proto=TCP chksum=0x783c src=192.168.5.21 dst=66.35.250.151 |<TCP
dataofs=5L chksum=0xbb39 options=[] |<Raw load='GET /index.html HTTP/1.0 \n\n'
|>>>>
```

Lecture des PCAP

OTERIA CYBER SCHOOL

```
>>>a=rdpcap("/spare/captures/isakmp.cap")
>>>a
<isakmp.cap: UDP:721 TCP:0 ICMP:0 Other:0>
```

Générer plusieurs paquets

OTERIA CYBER SCHOOL

```
>>>a=IP(dst="www.oteria.fr/30")
>>>a
<IP dst=Net(oteria.fr/30') |>
>>>[p for p in a]
[<IP dst=66.35.250.148 |>, <IP dst=66.35.250.149 |>, <IP dst=66.35.250.150 |>, <IP dst=66.35.250.151 |>]
>>>b=IP(ttl=[1,2,(5,9)])
>>>b
<IP ttl=[1, 2, (5, 9)] |>
>>>[p for p in b]
[<IP ttl=1 |>, <IP ttl=2 |>, <IP ttl=5 |>, <IP ttl=6 |>, <IP ttl=7 |>, <IP ttl=8 |>, <IP ttl=9 |>]
>>>c=TCP(dport=[80,443])
>>>[p for p in a/c]
[<IP frag=0 proto=TCP dst=66.35.250.148 |<TCP dport=80 |>>,
<IP frag=0 proto=TCP dst=66.35.250.148 |<TCP dport=443 |>>,
<IP frag=0 proto=TCP dst=66.35.250.149 |<TCP dport=80 |>>,
<IP frag=0 proto=TCP dst=66.35.250.149 |<TCP dport=443 |>>,
<IP frag=0 proto=TCP dst=66.35.250.150 |<TCP dport=80 |>>,
<IP frag=0 proto=TCP dst=66.35.250.150 |<TCP dport=443 |>>,
<IP frag=0 proto=TCP dst=66.35.250.151 |<TCP dport=80 |>>,
<IP frag=0 proto=TCP dst=66.35.250.151 |<TCP dport=443 |>>]
```

Commandes

OTERIA CYBER SCHOOL

Command	Effect
summary()	displays a list of summaries of each packet
nsummary()	same as previous, with the packet number
conversations()	displays a graph of conversations
show()	displays the preferred representation (usually nsummary())
filter()	returns a packet list filtered with a lambda function
hexdump()	returns a hexdump of all packets
hexraw()	returns a hexdump of the Raw layer of all packets
padding()	returns a hexdump of packets with padding
nzpadding()	returns a hexdump of packets with non-zero padding
plot()	plots a lambda function applied to the packet list
make_table()	displays a table according to a lambda function

Envoyer des paquets

send(IP(dst="1.2.3.4")/ICMP())

.

Sent 1 packets.

sendp(Ether()/IP(dst="1.2.3.4",ttl=(1,4)), iface="eth1") .... Sent 4 packets. sendp("I'm travelling on Ethernet", iface="eth1", loop=1, inter=0.2) ....^C Sent 16 packets. sendp(rdpcap("/tmp/pcapfile")) # tcpreplay ..... Sent 11 packets. Returns packets sent by send() send(IP(dst='127.0.0.1'), return_packets=True) . Sent 1 packets. <PacketList: TCP:0 UDP:0 ICMP:0 Other:1>

Fuzzing

La fonction fuzz() est capable de changer toute valeur par défaut qui ne doit pas être calculée (comme les sommes de contrôle) par un objet dont la valeur est aléatoire et dont le type est adapté au champ. Cela permet de construire rapidement des modèles de fuzzing et de les envoyer en boucle. Dans l'exemple suivant, la couche IP est normale et les couches UDP et NTP sont fuzzés. La somme de contrôle UDP sera correcte, le port de destination UDP sera surchargé par NTP à 123 et la version NTP sera forcée à 4. Tous les autres ports seront randomisés. Remarque : Si vous utilisez fuzz() dans la couche IP, les paramètres src et dst ne seront pas aléatoires, donc pour ce faire, utilisez RandIP(). :

```
send(IP(dst="target")/fuzz(UDP()/NTP(version=4)),loop=1)

.....^C

Sent 16 packets.
```

Envoyer et recevoir des paquets



>>>p = sr1(IP(dst="www.oteria.fr")/ICMP()/"XXXXXXXXXX")

Begin emission:

...Finished to send 1 packets.

.*

Received 5 packets, got 1 answers, remaining 0 packets

>>>p

<IP version=4L ihl=5L tos=0x0 len=39 id=15489 flags= frag=0L ttl=42 proto=ICMP chksum=0x51dd src=66.35.250.151 dst=192.168.5.21 options='' |<ICMP type=echo-reply code=0 chksum=0xee45 id=0x0 seq=0x0 |<Raw load='XXXXXXXXXX' |<Padding load='\x00\x00\x00\x00' |>>>>

Envoyer et recevoir des paquets

```
>>>p.show()
---[ IP ]---
version = 4L
ihl = 5L
tos = 0x0
len = 39
id = 15489
flags = frag = 0L
ttl = 42
proto = ICMP
chksum = 0x51dd
src = 66.35.250.151
dst = 192.168.5.21
options = ''
 ---[ ICMP ]---
                          type = echo-reply
                          code = 0
                          chksum = 0xee45
                          id = 0x0
                          seq = 0x0
 ---[ Raw ]---
                          load = 'XXXXXXXXXX'
---[ Padding ]---
                          load = '\x00\x00\x00\x00'
```

OTERIA CYBER SCHOOL

Requete DNS

OTERIA CYBER SCHOOL

```
sr1(IP(dst="192.168.5.1")/UDP()/DNS(rd=1,qd=DNSQR(qname="www.oteria.fr")))

Begin emission:

Finished to send 1 packets.

..*

Received 3 packets, got 1 answers, remaining 0 packets

<IP version=4L ihl=5L tos=0x0 len=78 id=0 flags=DF frag=0L ttl=64 proto=UDP
chksum=0xaf38 src=192.168.5.1 dst=192.168.5.21 options='' |<UDP sport=53 dport=53
len=58 chksum=0xd55d |<DNS id=0 qr=1L opcode=QUERY aa=0L tc=0L rd=1L ra=1L z=0L
rcode=ok qdcount=1 ancount=1 nscount=0 arcount=0 qd=<DNSQR
qname='www.slashdot.org.' qtype=A qclass=IN |> an=<DNSRR rrname='www.slashdot.org.'
type=A rclass=IN ttl=3560L rdata='66.35.250.151' |> ns=0 ar=0 |<Padding
load='\xc6\x94\xc7\xeb' |>>>>
```

Envoyer et recevoir des paquets

OTERIA CYBER SCHOOL

```
>>>sr(IP(dst="192.168.8.1")/TCP(dport=[21,22,23]))
Received 6 packets, got 3 answers, remaining 0 packets
(<Results: UDP:0 TCP:3 ICMP:0 Other:0>, <Unanswered: UDP:0 TCP:0 ICMP:0 Other:0>)
>>>ans, unans = _
>>>ans.summary()
IP / TCP 192.168.8.14:20 > 192.168.8.1:21 S ==> Ether / IP / TCP 192.168.8.1:21 > 192.168.8.14:20 RA / Padding
IP / TCP 192.168.8.14:20 > 192.168.8.1:22 S ==> Ether / IP / TCP 192.168.8.1:22 > 192.168.8.14:20 RA / Padding
IP / TCP 192.168.8.14:20 > 192.168.8.1:23 S ==> Ether / IP / TCP 192.168.8.1:23 > 192.168.8.14:20 RA / Padding
```

SYN scan

OTERIA CYBER SCHOOL

>>>sr1(IP(dst="72.14.207.99")/TCP(dport=80,flags="S"))

Begin emission:

.Finished to send 1 packets.

*

Received 2 packets, got 1 answers, remaining 0 packets

<IP version=4L ihl=5L tos=0x20 len=44 id=33529 flags= frag=0L ttl=244 proto=TCP chksum=0x6a34 src=72.14.207.99 dst=192.168.1.100 options=// | <TCP sport=www dport=ftp-data seq=2487238601L ack=1 dataofs=6L reserved=0L flags=SA window=8190 chksum=0xcdc7 urgptr=0 options=[('MSS', 536)] | <Padding load='V\xf7' |>>>

TCP traceroute




```
ans, unans = sr(IP(dst=target, ttl=(4,25),id=RandShort())/TCP(flags=0x2))
 *****.******.*.***..*.**Finished to send 22 packets.
***.....
Received 33 packets, got 21 answers, remaining 1 packets
>>>for snd,rcv in ans:
...               print snd.ttl, rcv.src, isinstance(rcv.payload, TCP)
...
4 194.250.107.181 0
5 194.51.159.65 0
6 194.51.159.49 0
7 193.251.126.34 0
8 193.251.126.154 0
9 193.251.241.89 0
10 193.251.241.110 0
...
```

Connaitre les fonctions :



>>>lsc()

sr : Send and receive packets at layer 3

sr1 : Send packets at layer 3 and return only the first answer

srp : Send and receive packets at layer 2 srp1 :

Send and receive packets at layer 2 and return only the first answer

srloop : Send a packet at layer 3 in loop and print the answer each time

srploop : Send a packet at layer 2 in loop and print the answer each time

sniff : Sniff packets

p0f : Passive OS

fingerprinting: which OS emitted this TCP SYN ?

arpcachepoison : Poison target's cache with (your MAC,victim's IP) couple

[...]

Sniffing

OTERIA CYBER SCHOOL

```
>>>sniff(filter="icmp and host 66.35.250.151", count=2)
<Sniffed: UDP:0 TCP:0 ICMP:2 Other:0>
>>>a=_
>>>a.nsummary()
0000 Ether / IP / ICMP 192.168.5.21 echo-request 0 / Raw
0001 Ether / IP / ICMP 192.168.5.21 echo-request 0 / Raw
```

Filtres




```
from scapy.all import *
def maFonction(paquetRecu):
        print ("j'ai recu un paquet »)

sniff(iface="eth0", prn=maFonction, count=15)
```

Scapy va sniffer sur l'interface "eth0" au maximum 15 paquets et pour chaque paquets reçu, la fonction "maFonction" sera exécutée.

Filtres

Pour obtenir quelques informations sur le paquet sniffé il suffit de faire appel à la méthode summary()

```
from scapy.all import *
def maFonction(paquetRecu):
        print paquetRecu.summary()
sniff(iface="eth0", prn=maFonction, count=15)
```

Scapy va sniffer sur l'interface "eth0" au maximum 15 paquets et pour chaque paquets reçu la fonction "maFonction" sera exécutée.

Filtres

Pour obtenir le détail de toute les couches c'est la méthode show() qu'il faut utiliser. Notez qu'il n'y a plus besoin du print dans ce cas la

```
from scapy.all import *
def maFonction(paquetRecu):
        paquetRecu.show()
sniff(iface="eth0", prn=maFonction, count=15)
```

Scapy va sniffer sur l'interface "eth0" au maximum 15 paquets et pour chaque paquets reçu la fonction "maFonction" sera exécutée.

Filtres

Utilisation des filtres BPF, Berkeley Packet Filter (rapides/pas flexibles) :

Syntaxe : [Protocole] [Direction] [Type]

- Protocole peut être :
  - *ether, wlan, ip, ip6, arp, rarp, tcp, udp, icmp*
- Direction peut être
  - *src, dst, src or dst, src and dst*
- Type peut être host, net, port. Le type par défaut est host

La syntaxe n'est pas aisée à prendre en main, pour plus d'infos :

- *http://biot.com/capstats/bpf.html*
- *http://www.cs.ucr.edu/~marios/ethereal-tcpdump.pdf*
- *http://fr.wikipedia.org/wiki/BSD_Packet_Filter*

Filtres

Exemple de filtres BPF :

- "tcp" => N'accepte que les paquets TCP
- "tcp or udp" => N'accepte que les paquets TCP ou UDP
- "ip and host 192.168.1.1" => N'accepte que les paquets ip depuis ou vers 192.168.1.1
- "ip and tcp 192.168.1.1" => N'accepte que les paquets ip depuis ou vers 192.168.1.1
- "ip and tcp and src net 192.168.1.0/24" => N'accepte que les paquets tcp/ip depuis 192.168.1.0/24 (host ne permet pas la notation CIDR)
- "tcp dst port 443" => N'accepte que les paquets tcp à destination du port 443

Lfilter




```
from scapy.all import *
def monFiltre(paquetRecu):
        if paquetRecu.haslayer(TCP):
                return True
        else:
                return False
def maFonction(paquetRecu):
        print "j'ai recu un paquet"
sniff(iface="eth0", prn=maFonction, lfilter=monFiltre, count=15)
```

Scapy va sniffer sur l'interface "eth0" au maximum 15 paquets et pour chaque paquets reçu la fonction "maFonction" sera exécutée si le paquet est de type TCP.

Accéder aux variables

Que ca soit dans une fonction de filtrage ou une fonction de traitement il est bien souvent utile d'accéder aux informations du paquet comme l'IP source.

Pour ce faire, il suffit d'écrire le nom de la couche désirée entre crochets : ls(paquetRecu[IP]) affichera les informations du paquet à partir de la couche IP.

par exemple :

```
def maFonction(paquetRecu):
        ls(paquetRecu[IP])
```

Si dans vous voulez accéder à un champ particulier, il suffit de rajouter le nom de ce champ , précédé par un point après les crochets délimitant la couche

```
def maFonction(paquetRecu):
        print paquetRecu[IP].src
```

Cette fonctionne affiche l'ip source du paquet reçu en argument.

lfilter

Dans le cas ou une couche n'est pas connue (comme HTTP par exemple), le contenu de cette couche est situé dans la couche appelée Raw (brute). Le programme suivant affiche les requêtes générées par notre ordinateur lors de la visite d'un site web :

```
from scapy.all import *
def monFiltre(pkt):
        if pkt.haslayer(Raw) and pkt[IP].src == "192.168.1.169" and pkt[TCP].dport==80:
                return True
        else:
                return False
def maFonction(paquetRecu):
        print (paquetRecu[Raw])


sniff(iface="eth0", prn=maFonction, lfilter=monFiltre, count=15)
```

Sniffing + OSfingerprinting



>>>p

<Ether dst=00:10:4b:b3:7d:4e src=00:40:33:96:7b:60 type=0x800 |<IP version=4L ihl=5L tos=0x0 len=60 id=61681 flags=DF frag=0L ttl=64 proto=TCP chksum=0xb85e src=192.168.8.10 dst=192.168.8.1 options='' |<TCP sport=46511 dport=80 seq=2023566040L ack=0L dataofs=10L reserved=0L flags=SEC window=5840 chksum=0x570c urgptr=0 options=[('Timestamp', (342940201L, 0L)), ('MSS', 1460), ('NOP', ()), ('SAckOK', ''), ('WScale', 0)] |>>>

>>>load_module("p0f")

>>>p0f(p)

(1.0, ['Linux 2.4.2 - 2.4.14 (1)’])

>>>a=sniff(prn=prnp0f)

(1.0, ['Linux 2.4.2 - 2.4.14 (1)’])

(1.0, ['Linux 2.4.2 - 2.4.14 (1)’])

(0.875, ['Linux 2.4.2 - 2.4.14 (1)', 'Linux 2.4.10 (1)', 'Windows 98 (?)’])

(1.0, ['Windows 2000 (9)'])